

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Code of Conduct:

I janani.k(192511243) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: janani.k

ANALYTICAL QUESTION

1. Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given

Employee and Department relations to retrieve required records.

Aim

Apply relational algebra operations on Employee and Department.

Sample Table

EmpID	EmpName	DeptID	Salary
101	Hari	D1	60000
102	Arun	D2	45000
103	Priya	D1	70000
104	Kavin	D3	55000

Solution

$\sigma_{\text{Salary} > 50000}(\text{Employee})$
 $\pi_{\text{EmpName}}(\text{Employee})$
 $\text{Employee} \bowtie \text{Department}$
 $\text{Employee} \times \text{Department}$

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

Aim

Write SQL DDL statements for STUDENT and DEPARTMENT with constraints.

Sample Table(s)

STUDENT(StudentID, Name, DOB, DeptID)

DEPARTMENT(DeptID, DeptName)

Solution

```
CREATE TABLE Department(  
  DeptID INT PRIMARY KEY,  
  DeptName VARCHAR(30) NOT NULL);
```

```
CREATE TABLE Student(  
  StudentID INT PRIMARY KEY,  
  Name VARCHAR(50) NOT NULL,  
  DOB DATE,  
  DeptID INT,  
  FOREIGN KEY(DeptID) REFERENCES Department(DeptID));
```

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

3. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

Aim

GROUP BY, HAVING and aggregate functions.

Sample Table(s)

SALES(SaleID, Product, Amount, Region)

Solution

```
SELECT Region,COUNT(*),SUM(Amount),AVG(Amount),MAX(Amount),MIN(Amount)  
FROM Sales  
GROUP BY Region  
HAVING SUM(Amount)>100000;
```

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

4. Write a correlated sub-query to find employees earning more than the average salary of their own department.**Aim**

Correlated subquery for employees earning above department average.

Sample Table(s)

EmpID	EmpName	DeptID	Salary
101	Hari	D1	60000
102	Arun	D2	45000
103	Priya	D1	70000
104	Kavin	D3	55000

Solution

```
SELECT e.*  
FROM Employee e  
WHERE Salary > (  
SELECT AVG(Salary)  
FROM Employee  
WHERE DeptID=e.DeptID);
```

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.**Aim**

Demonstrate INNER, LEFT, RIGHT, FULL and CROSS JOIN.

Sample Table(s)

EmpID	EmpName	DeptID	Salary
101	Hari	D1	60000
102	Arun	D2	45000
103	Priya	D1	70000
104	Kavin	D3	55000

Solution

Provide SQL examples for INNER, LEFT, RIGHT, FULL OUTER and CROSS JOIN using Employee and Project tables.

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

6. Create a view “DeptSalaryView”; that displays department name, total employees, and average salary. Write a query on this view.

Aim

Create DeptSalaryView and query it.

Sample Table(s)

EmpID	EmpName	DeptID	Salary
101	Hari	D1	60000
102	Arun	D2	45000
103	Priya	D1	70000
104	Kavin	D3	55000

Solution

```
CREATE VIEW DeptSalaryView AS
SELECT DeptID,COUNT(*) AS TotalEmployees,AVG(Salary) AS AvgSalary
FROM Employee GROUP BY DeptID;
```

```
SELECT * FROM DeptSalaryView;
```

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

7. Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

Aim

Relational algebra: students enrolled in all courses.

Sample Table(s)

STUDENT, COURSE, ENROLLMENT / AIRLINE tables as required.

Solution
$$\pi_{\text{StudentName}}(\text{Student}) \div \pi_{\text{CourseID}}(\text{Course})$$
Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

8. Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

Aim

SQL DML statements (INSERT, UPDATE, DELETE).

Sample Table(s)

STUDENT(StudentID, Name, DOB, DeptID)

DEPARTMENT(DeptID, DeptName)

Solution

INSERT INTO Student VALUES(1,'Hari','2004-01-01',1);

UPDATE Student SET Name='Haran' WHERE StudentID=1;

DELETE FROM Student WHERE StudentID=1;

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

9. Apply tuple relational calculus to express a query: Find all employees who work in the “IT”; department and earn more than 50000.

Aim

Tuple Relational Calculus query.

Sample Table(s)

EmpID	EmpName	DeptID	Salary
101	Hari	D1	60000
102	Arun	D2	45000
103	Priya	D1	70000
104	Kavin	D3	55000

Solution

{ t | Employee(t) ∧ t.Dept='IT' ∧ t.Salary>50000 }

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset.

10. Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

Aim

Design Airline Reservation schema and write five SQL queries.

Sample Table(s)

STUDENT, COURSE, ENROLLMENT / AIRLINE tables as required.

Solution

Schema:

Passenger(PassengerID,...)

Flight(FlightID,...)

Booking(BookingID,...)

Ticket(TicketID,...)

Example queries:

1. List all bookings.
2. Flights with available seats.
3. Passengers on a flight.
4. Total bookings per flight.
5. Create a view for confirmed bookings.

Explanation

The above solution satisfies the required DBMS concept using a self-generated sample dataset. Modify names or values if your faculty provides a specific dataset

